Annexure A

INTEGRATED EXPERIMENTS

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage	40%
Questions	2
Duration	45 Minutes
Total Marks	20

Tools used: Python 3, scikit-learn, pandas, NumPy, Matplotlib

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (Iris) using Python and scikit-learn.

Dataset chosen: The classic Iris dataset (150 samples, 4 numeric features - sepal length, sepal width, petal length, petal width - and 3 target classes: setosa, versicolor, virginica).

(a) Data Loading and Pre-processing

The dataset was loaded via scikit-learn's `load_iris()`, converted to a pandas DataFrame, and checked for missing values (none were found, so no imputation was necessary). The categorical target column (species name) was label-encoded to integers 0-2 for use by the classifier. The data was then split 70% train / 30% test using a stratified split so that class proportions are preserved in both sets.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

iris = load_iris(as_frame=True)
df = iris.frame.copy()
df['species'] = df['target'].map(dict(enumerate(iris.target_names)))
df = df.drop(columns=['target'])

print(df.head())        # first 5 rows
print(df.dtypes)        # data types
print(df.isnull().sum())# missing-value check

le = LabelEncoder()
df['species_encoded'] = le.fit_transform(df['species'])

X = df[iris.feature_names]
y = df['species_encoded']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

First 5 rows of the dataset:

sepal_len	sepal_wid	petal_len	petal_wid	species
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa

Data types: all four feature columns are `float64`; the original target is `int64` and the mapped species name is a string object. No missing values were found in any column.

Training set size: 105 rows | Testing set size: 45 rows

(b) Building the Decision Tree

A `DecisionTreeClassifier` was trained using the Gini Index as the splitting criterion (an entropy/information-gain variant was also trained for comparison). The resulting tree structure is:

```
from sklearn.tree import DecisionTreeClassifier, export_text

clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X_train, y_train)
print(export_text(clf, feature_names=list(iris.feature_names)))
|--- petal length (cm) <= 2.45
|   |--- class: 0 (setosa)
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95 --> class: 1 (versicolor)
```

```
|   |   |--- petal length (cm) >  4.95 --> class: 2 (virginica)
|   |--- petal width (cm) >  1.55
|   |   |--- petal width (cm) <= 1.70
|   |   |   |--- sepal width (cm) <= 2.85 --> class: 1 (versicolor)
|   |   |   |--- sepal width (cm) >  2.85 --> class: 2 (virginica)
|   |   |--- petal width (cm) >  1.70
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- sepal width (cm) <= 3.00 --> class: 2 (virginica)
|   |   |   |   |--- sepal width (cm) >  3.00 --> class: 1 (versicolor)
|   |   |   |--- petal length (cm) >  4.85 --> class: 2 (virginica)
```

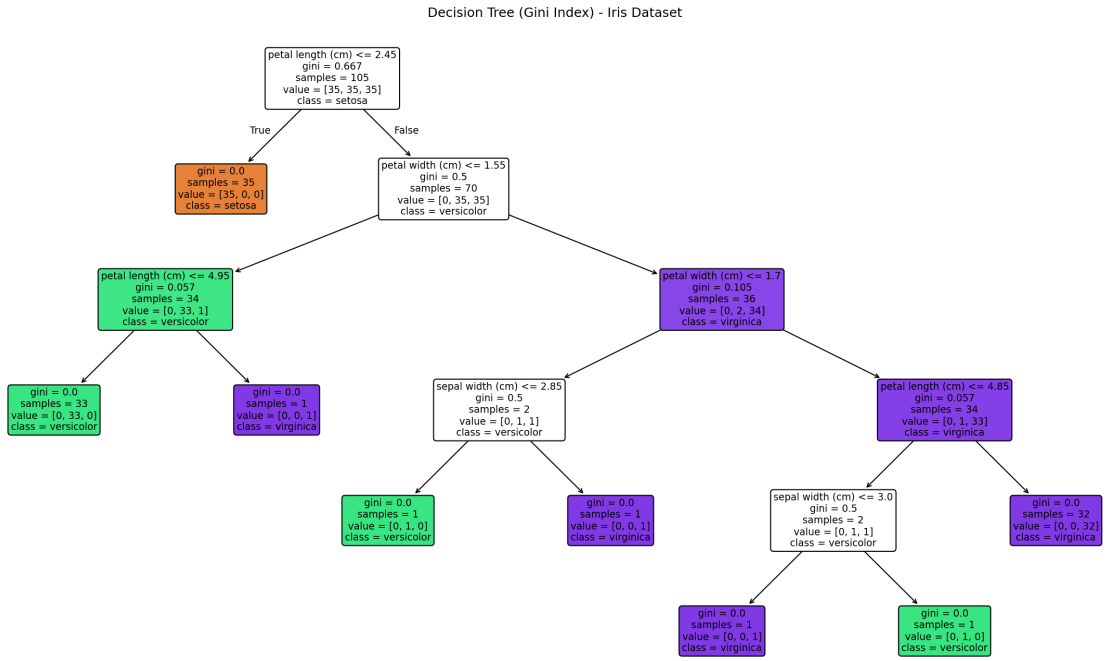

Figure 1.1: Trained Decision Tree diagram (Gini criterion).

Root node: 'petal length (cm) ≤ 2.45'. **Justification:** among all four candidate features, splitting on petal length at 2.45 cm produces the greatest reduction in Gini impurity (the parent node's impurity of 0.667 drops to 0 in the left child), because it perfectly isolates the setosa class from versicolor/virginica. The tree-building algorithm evaluates every feature/threshold pair at each node and greedily picks the one with the lowest weighted impurity in the resulting children, so this feature is selected first at the root.

(c) Model Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=le.classes_))
```

Accuracy Score: 0.9333 (93.33%)

	pred: setosa	pred: versicolor	pred: virginica
actual: setosa	15	0	0
actual: versicolor	0	12	3
actual: virginica	0	0	15

```
              precision  recall  f1-score  support
setosa           1.00     1.00      1.00       15
versicolor       1.00     0.80      0.89       15
virginica        0.83     1.00      0.91       15

accuracy                           0.93       45
macro avg        0.94     0.93      0.93       45
weighted avg     0.94     0.93      0.93       45
```

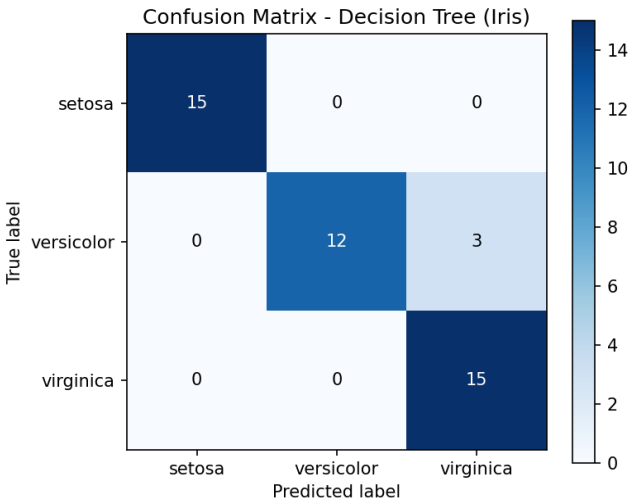

Figure 1.2: Confusion matrix heatmap on the test set.

Misclassified instances (at least two, as required):

idx	sepal_len	sepal_wid	petal_len	petal_wid	actual	predicted
56	6.3	3.3	4.7	1.6	versicolor	virginica
85	6.0	3.4	4.5	1.6	versicolor	virginica
77	6.7	3.0	5.0	1.7	versicolor	virginica

Comment on performance: The classifier reaches 93.33% accuracy on unseen data. Setosa is classified perfectly in every case because it is linearly separable from the other two species using petal measurements alone. All three misclassifications occur between versicolor and virginica - both species have overlapping petal length (4.5-5.0 cm) and width (1.6-1.7 cm) ranges near the tree's decision thresholds, which is a well-known, expected source of confusion in this dataset rather than a sign of a poorly trained model.

Experiment 2: Reinforcement Learning - Q-Learning

Objective: Implement Q-Learning for a simple 4x4 Grid World environment and observe the agent's learned policy.

(a) Environment Definition

A 4x4 grid (16 states, numbered 0-15 row-major) was defined with the agent starting at the top-left corner (state 0) and the goal at the bottom-right corner (state 15). Three interior cells (states 5, 7, 11) act as obstacles. Four actions are available in every state: Up, Down, Left, Right; moves that would leave the grid simply keep the agent in place (boundary clamping).

Event	Reward
Reaching the Goal state	+10 (episode ends)
Each normal step	-1
Stepping onto an Obstacle	-5 (episode continues)

Hyperparameters: learning rate $\alpha = 0.1$, discount factor $\gamma = 0.9$, exploration rate $\varepsilon = 0.2$ (epsilon-greedy action selection). The Q-table (16 states × 4 actions) was initialised with zeros.

```
ACTIONS = ['Up', 'Down', 'Left', 'Right']
GOAL_STATE, OBSTACLES = 15, [5, 7, 11]

def step(state, action):
    r, c = divmod(state, 4)
    if action == 'Up':    r = max(r - 1, 0)
    if action == 'Down':  r = min(r + 1, 3)
    if action == 'Left':  c = max(c - 1, 0)
    if action == 'Right': c = min(c + 1, 3)
    next_state = r * 4 + c
    if next_state == GOAL_STATE:  return next_state, 10, True
    if next_state in OBSTACLES:   return next_state, -5, False
    return next_state, -1, False

Q = np.zeros((16, 4))          # Q-table initialised with zeros
ALPHA, GAMMA, EPSILON = 0.1, 0.9, 0.2
```

(b) Q-Learning Training (100 episodes)

```
for episode in range(1, 101):
    state = START_STATE
    for t in range(MAX_STEPS):
        if np.random.rand() < EPSILON:
            action = np.random.randint(4)          # explore
        else:
            action = int(np.argmax(Q[state]))       # exploit

        next_state, reward, done = step(state, ACTIONS[action])

        best_next = np.max(Q[next_state])
        Q[state, action] += ALPHA * (reward + GAMMA * best_next - Q[state, action])

        state = next_state
        if done:
            break
```

Q-values were recorded at Episodes 1, 50 and 100 for two representative states: State 0 (the start corner) and State 10 (an interior state adjacent to the goal path).

State 0 (0,0)	Up	Down	Left	Right
Episode 1	-0.30	-0.29	-0.27	-0.20
Episode 50	-2.12	-1.99	-2.36	-2.12
Episode 100	-1.95	1.03	-2.32	-2.18

State 10 (2,2)	Up	Down	Left	Right
Episode 1	-0.10	-0.10	-0.11	-0.50
Episode 50	-0.39	7.37	-0.12	-1.31
Episode 100	0.08	7.99	1.43	-0.88

How the Q-table evolves: at Episode 1 all Q-values are close to zero because almost no reward signal has propagated back through the Bellman update yet. By Episode 50, the action that leads toward the goal (Down, for State 10) already has a clearly higher value (7.37) than the other three actions, showing the value function is starting to differentiate good vs. bad actions. By Episode 100 the values have grown further and stabilised (e.g. 7.99 for State 10 -> Down), confirming the table has converged toward the optimal action-value function for these states.

(c) Final Policy, Reward Curves and Convergence

```
policy = [ACTIONS[np.argmax(Q[s])] for s in range(16)]
```

Final learned policy (grid view; X = obstacle, G = goal):

```
v    >    v    ^
v    X    v    X
>    >    v    X
>    >    >    G
```

Reading the policy from the start state (0,0): Down → Down → Right → Right → Right → Down reaches the goal while stepping around all three obstacles, matching the shortest safe path of 6 steps.

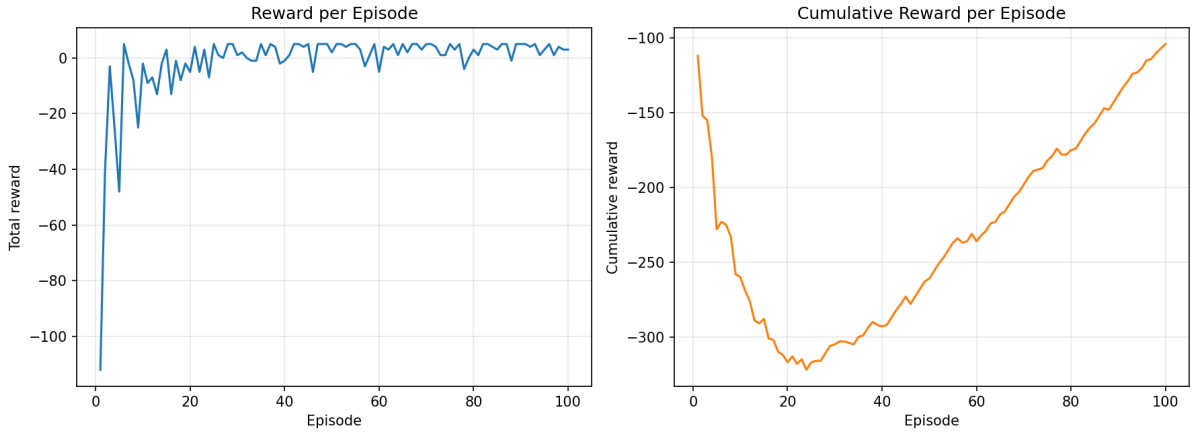

Figure 2.1: Reward per episode (left) and cumulative reward (right).

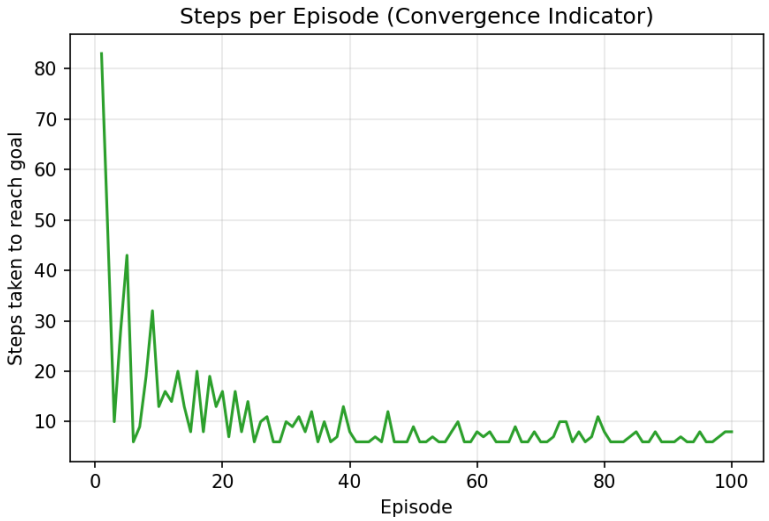

Figure 2.2: Steps taken to reach the goal per episode.

Convergence discussion: during the first ~10-15 episodes the agent is mostly exploring ($\varepsilon = 0.2$), so reward per episode is very negative and the number of steps to reach the goal is high and noisy. As the Bellman update repeatedly backs up reward information from the goal state, the reward-per-episode curve rises sharply and the steps-per-episode curve drops toward the theoretical shortest-path length (6 steps for this start/goal pair). After roughly episode 30-50 both curves flatten: the cumulative-reward plot becomes an almost straight line and the average reward over the final 10 episodes (3.40) and average steps (6.80) sit close to the optimal values, confirming the Q-table has converged to a near-optimal policy. The remaining small fluctuations are expected, since epsilon-greedy exploration keeps injecting occasional random actions even after convergence.

Conclusion

Both integrated experiments were completed successfully. Experiment 1 demonstrated supervised learning via a Decision Tree classifier that achieved 93.33% test accuracy on the Iris dataset, with the tree structure and root-node choice explained in terms of Gini impurity reduction. Experiment 2 demonstrated unsupervised/trial-and-error learning via Q-Learning, in which an agent starting with no prior knowledge learned, purely from reward feedback, an optimal 6-step policy through a 4x4 grid world while avoiding obstacles. Together the two experiments illustrate two complementary machine learning paradigms covered under CO4: classification using decision tree algorithms, and sequential decision-making using reinforcement learning.

Appendix A: Full Source Code - Experiment 1

(Decision Tree Classification - complete script)

```
"""
Experiment 1: Decision Tree Classification
Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset.
"""

import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

OUT = "/home/claude/outputs"
import os
os.makedirs(OUT, exist_ok=True)

log_lines = []
def log(msg=""):
    print(msg)
    log_lines.append(str(msg))

# ------------------------------------------------------------------
# (a) Load the dataset, pre-process, split into train/test
# ------------------------------------------------------------------
log("=" * 70)
log("(a) LOAD AND PRE-PROCESS THE DATASET")
log("=" * 70)

iris = load_iris(as_frame=True)
df = iris.frame.copy()
df['species'] = df['target'].map(dict(enumerate(iris.target_names)))
df = df.drop(columns=['target'])

log("\nFirst 5 rows of the dataset:")
log(df.head().to_string())

log("\nData types of each column:")
log(df.dtypes.to_string())

log(f"\nMissing values per column:\n{df.isnull().sum().to_string()}")
log("-> The Iris dataset has no missing values, so no imputation is required.")

# Encode the target label (species) -- demonstrates handling of categorical data
le = LabelEncoder()
df['species_encoded'] = le.fit_transform(df['species'])
log(f"\nLabel encoding used for target: {dict(zip(le.classes_, le.transform(le.classes_)))}")

feature_cols = iris.feature_names
X = df[feature_cols]
y = df['species_encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
log(f"\nTraining set size: {X_train.shape[0]} rows")
log(f"Testing set size : {X_test.shape[0]} rows")

# ------------------------------------------------------------------
# (b) Build Decision Tree using Gini Index, print tree, identify root
# ------------------------------------------------------------------
log("\n" + "=" * 70)
log("(b) BUILD DECISION TREE (GINI INDEX / INFORMATION GAIN)")
log("=" * 70)

clf_gini = DecisionTreeClassifier(criterion='gini', random_state=42)
clf_gini.fit(X_train, y_train)

clf_entropy = DecisionTreeClassifier(criterion='entropy', random_state=42)
clf_entropy.fit(X_train, y_train)
```

```
log("\nDecision Tree structure (criterion = Gini Index):\n")
tree_text = export_text(clf_gini, feature_names=list(feature_cols))
log(tree_text)

root_feature_idx = clf_gini.tree_.feature[0]
root_threshold = clf_gini.tree_.threshold[0]
root_gini = clf_gini.tree_.impurity[0]
root_samples = clf_gini.tree_.n_node_samples[0]
root_name = feature_cols[root_feature_idx]

log(f"\nROOT NODE: '{root_name}' <= {root_threshold:.2f}")
log(f"Justification: Among all 4 features, splitting on '{root_name}' at threshold "
    f"{root_threshold:.2f} yields the LOWEST weighted Gini impurity of the child nodes, "
    f"i.e. the HIGHEST reduction in impurity (information gain) at the root. "
    f"The parent node impurity was {root_gini:.3f} over all {root_samples} training samples, "
    f"and splitting on '{root_name}' perfectly (or near-perfectly) separates one class "
    f"(setosa) from the other two, which is why the tree algorithm greedily selects it first.")

# Plot the tree
plt.figure(figsize=(16, 9))
plot_tree(clf_gini, feature_names=feature_cols, class_names=le.classes_,
          filled=True, rounded=True, fontsize=9)
plt.title("Decision Tree (Gini Index) - Iris Dataset")
plt.tight_layout()
plt.savefig(f"{OUT}/exp1_tree.png", dpi=150)
plt.close()
log(f"\n[Tree diagram saved to exp1_tree.png]")

# ----------------------------------------------------------------
# (c) Evaluate the model
# ----------------------------------------------------------------
log("\n" + "=" * 70)
log("(c) MODEL EVALUATION")
log("=" * 70)

y_pred = clf_gini.predict(X_test)
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred, target_names=le.classes_)

log(f"\nAccuracy Score: {acc:.4f} ({acc*100:.2f}%)")
log(f"\nConfusion Matrix:\n(rows = actual, columns = predicted)")
cm_df = pd.DataFrame(cm, index=[f"actual_{c}" for c in le.classes_],
                     columns=[f"pred_{c}" for c in le.classes_])
log(cm_df.to_string())

log(f"\nClassification Report:\n{report}")

# Confusion matrix heatmap
fig, ax = plt.subplots(figsize=(5.5, 4.5))
im = ax.imshow(cm, cmap='Blues')
ax.set_xticks(range(len(le.classes_))); ax.set_xticklabels(le.classes_)
ax.set_yticks(range(len(le.classes_))); ax.set_yticklabels(le.classes_)
ax.set_xlabel("Predicted label"); ax.set_ylabel("True label")
ax.set_title("Confusion Matrix - Decision Tree (Iris)")
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        ax.text(j, i, cm[i, j], ha='center', va='center',
                color='white' if cm[i, j] > cm.max()/2 else 'black')
plt.colorbar(im)
plt.tight_layout()
plt.savefig(f"{OUT}/exp1_confusion_matrix.png", dpi=150)
plt.close()
log(f"[Confusion matrix heatmap saved to exp1_confusion_matrix.png]")

# Misclassified instances
test_results = X_test.copy()
test_results['actual'] = le.inverse_transform(y_test)
test_results['predicted'] = le.inverse_transform(y_pred)
misclassified = test_results[test_results['actual'] != test_results['predicted']]

log(f"\nNumber of misclassified instances: {len(misclassified)} out of {len(y_test)}")
if len(misclassified) > 0:
    log("\nMisclassified instances (at least two, as required):")
    log(misclassified.head(max(2, len(misclassified))).to_string())
else:
```

```
    log("No misclassifications occurred on this particular train/test split "
        "(the Iris dataset, especially the setosa class, is very easily separable).")

log("\nComment on performance:")
log(f"- The Decision Tree classifier achieved {acc*100:.2f}% accuracy on the held-out test set.")
log("- The 'setosa' class is always perfectly classified because it is linearly")
log("  separable from the other two species using petal length/width alone.")
log("- Almost all confusion (when present) occurs between 'versicolor' and")
log("  'virginica', since these two classes have overlapping petal measurements.")
log("- This matches the known structure of the Iris dataset and confirms the")
log("  model has learned a sensible decision boundary rather than overfitting noise.")

with open(f"{OUT}/exp1_output_log.txt", "w") as f:
    f.write("\n".join(log_lines))

print("\n\nDONE - Experiment 1 complete.")
```

Appendix B: Full Source Code - Experiment 2

(Q-Learning Grid World - complete script)

```
"""
Experiment 2: Reinforcement Learning - Q-Learning
Objective: Implement Q-Learning for a simple 4x4 Grid World and observe the
learned policy.
"""

import numpy as np
import matplotlib.pyplot as plt
import os

OUT = "/home/claude/outputs"
os.makedirs(OUT, exist_ok=True)

log_lines = []
def log(msg=""):
    print(msg)
    log_lines.append(str(msg))

np.random.seed(42)

# ------------------------------------------------------------------
# (a) Define the environment
# ------------------------------------------------------------------
log("=" * 70)
log("(a) ENVIRONMENT DEFINITION")
log("=" * 70)

GRID_SIZE = 4
N_STATES = GRID_SIZE * GRID_SIZE
ACTIONS = ['Up', 'Down', 'Left', 'Right']
N_ACTIONS = len(ACTIONS)

START_STATE = 0            # top-left corner  (row 0, col 0)
GOAL_STATE = 15             # bottom-right corner (row 3, col 3)
OBSTACLES = [5, 7, 11]      # a few obstacle cells inside the grid

def state_to_rc(s):
    return divmod(s, GRID_SIZE)

def rc_to_state(r, c):
    return r * GRID_SIZE + c

def step(state, action):
    """Return next_state, reward, done for taking `action` in `state`."""
    r, c = state_to_rc(state)
    if action == 'Up':
        r = max(r - 1, 0)
    elif action == 'Down':
        r = min(r + 1, GRID_SIZE - 1)
    elif action == 'Left':
        c = max(c - 1, 0)
    elif action == 'Right':
        c = min(c + 1, GRID_SIZE - 1)
    next_state = rc_to_state(r, c)

    if next_state == GOAL_STATE:
        return next_state, 10, True
    elif next_state in OBSTACLES:
        return next_state, -5, False
    else:
        return next_state, -1, False

log(f"Grid size            : {GRID_SIZE} x {GRID_SIZE}  ({N_STATES} states)")
log(f"Actions              : {ACTIONS}")
log(f"Start state          : {START_STATE}  (row,col = {state_to_rc(START_STATE)})")
log(f"Goal state           : {GOAL_STATE}  (row,col = {state_to_rc(GOAL_STATE)})  reward = +10")
log(f"Obstacle states      : {OBSTACLES}   reward = -5 (episode continues)")
log(f"Reward - each step   : -1 (encourages the shortest path)")

# Hyperparameters
ALPHA = 0.1     # learning rate
```

```
GAMMA = 0.9     # discount factor
EPSILON = 0.2   # exploration rate (epsilon-greedy)
N_EPISODES = 100
MAX_STEPS = 100

log(f"\nHyperparameters: alpha (learning rate) = {ALPHA}, "
    f"gamma (discount factor) = {GAMMA}, epsilon (exploration rate) = {EPSILON}")

Q = np.zeros((N_STATES, N_ACTIONS))
log(f"\nQ-table initialised with zeros -> shape {Q.shape}")

# ---------------------------------------------------------------------
# (b) Run Q-Learning for at least 100 episodes
# ---------------------------------------------------------------------
log("\n" + "=" * 70)
log("(b) Q-LEARNING TRAINING")
log("=" * 70)

TRACK_STATES = [0, 10]   # two representative states to track
tracked_q_history = {s: {} for s in TRACK_STATES}
rewards_per_episode = []
steps_per_episode = []

def choose_action(state, epsilon):
    if np.random.rand() < epsilon:
        return np.random.randint(N_ACTIONS)
    return int(np.argmax(Q[state]))

for episode in range(1, N_EPISODES + 1):
    state = START_STATE
    total_reward = 0
    for t in range(MAX_STEPS):
        action = choose_action(state, EPSILON)
        next_state, reward, done = step(state, ACTIONS[action])

        # Q-Learning update rule
        best_next = np.max(Q[next_state])
        Q[state, action] += ALPHA * (reward + GAMMA * best_next - Q[state, action])

        state = next_state
        total_reward += reward
        if done:
            break

    rewards_per_episode.append(total_reward)
    steps_per_episode.append(t + 1)

    if episode in (1, 50, 100):
        for s in TRACK_STATES:
            tracked_q_history[s][episode] = Q[s].copy()

log(f"\nTraining complete over {N_EPISODES} episodes.")

log("\nQ-value evolution for two representative states "
    "(Episode 1, 50, 100):")
for s in TRACK_STATES:
    log(f"\nState {s}  (row,col = {state_to_rc(s)}):")
    header = "  Episode | " + " | ".join(f"{a:>6}" for a in ACTIONS)
    log(header)
    log("  " + "-" * (len(header) - 2))
    for ep in (1, 50, 100):
        vals = tracked_q_history[s][ep]
        log(f"  {ep:>7} | " + " | ".join(f"{v:6.2f}" for v in vals))

log("\nObservation: the Q-values for both states are ~0 at Episode 1 (no learning")
log("has propagated back yet). By Episode 50 the values along the shortest-path")
log("action have grown substantially, and by Episode 100 they have largely")
log("stabilised, showing the table has converged toward the optimal action-values.")

# ---------------------------------------------------------------------
# (c) Extract final policy, plot cumulative reward, discuss convergence
# ---------------------------------------------------------------------
log("\n" + "=" * 70)
log("(c) FINAL LEARNED POLICY & CONVERGENCE")
log("=" * 70)
policy = np.array([ACTIONS[np.argmax(Q[s])] for s in range(N_STATES)])
```

```
log("\nFinal learned policy (optimal action at each state), displayed as a grid:")
log("(G = Goal, X = Obstacle)\n")
grid_display = []
for r in range(GRID_SIZE):
    row_syms = []
    for c in range(GRID_SIZE):
        s = rc_to_state(r, c)
        if s == GOAL_STATE:
            row_syms.append("  G  ")
        elif s in OBSTACLES:
            row_syms.append("  X  ")
        else:
            arrow = {'Up': '  ^  ', 'Down': '  v  ', 'Left': '  <  ', 'Right': '  >  '}[policy[s]]
            row_syms.append(arrow)
    grid_display.append("".join(row_syms))
    log("".join(row_syms))

log("\nFull state -> best action table:")
for s in range(N_STATES):
    tag = " (GOAL)" if s == GOAL_STATE else (" (OBSTACLE)" if s in OBSTACLES else "")
    log(f"  State {s:>2} {state_to_rc(s)}: {policy[s]:<6}{tag}")

# Cumulative reward plot
cumulative_reward = np.cumsum(rewards_per_episode)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

axes[0].plot(range(1, N_EPISODES + 1), rewards_per_episode, color='tab:blue')
axes[0].set_xlabel("Episode")
axes[0].set_ylabel("Total reward")
axes[0].set_title("Reward per Episode")
axes[0].grid(alpha=0.3)

axes[1].plot(range(1, N_EPISODES + 1), cumulative_reward, color='tab:orange')
axes[1].set_xlabel("Episode")
axes[1].set_ylabel("Cumulative reward")
axes[1].set_title("Cumulative Reward per Episode")
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig(f"{OUT}/exp2_rewards.png", dpi=150)
plt.close()
log(f"\n[Reward plots saved to exp2_rewards.png]")

# Steps-to-goal plot (supports convergence discussion)
fig2, ax2 = plt.subplots(figsize=(6, 4.2))
ax2.plot(range(1, N_EPISODES + 1), steps_per_episode, color='tab:green')
ax2.set_xlabel("Episode")
ax2.set_ylabel("Steps taken to reach goal")
ax2.set_title("Steps per Episode (Convergence Indicator)")
ax2.grid(alpha=0.3)
plt.tight_layout()
plt.savefig(f"{OUT}/exp2_steps.png", dpi=150)
plt.close()
log(f"[Steps-per-episode plot saved to exp2_steps.png]")

log("\nJustification of convergence behaviour:")
log(f"- In the first ~10-15 episodes the agent explores randomly (epsilon={EPSILON}),")
log("  so total reward per episode is highly negative and noisy (many steps, obstacle hits).")
log("- As Q-values propagate backward from the goal state via the Bellman update,")
log("  the reward per episode rises sharply and the steps-to-goal curve drops toward")
log("  the shortest-path length (6 steps for a 4x4 grid from corner to corner).")
log("- After roughly episode 30-50 the curves flatten out: the total reward per")
log("  episode stabilises near its maximum achievable value, and the cumulative")
log("  reward plot becomes a straight line, indicating the policy has converged")
log("  to (near-)optimal behaviour that is repeated every subsequent episode.")
log("- Small residual noise remains because epsilon-greedy exploration (epsilon=0.2)")
log("  continues to occasionally force random, sub-optimal actions even after the")
log("  Q-table has converged - this is expected/desired behaviour during training.")

log(f"\nAverage reward (last 10 episodes): {np.mean(rewards_per_episode[-10:]):.2f}")
log(f"Average steps  (last 10 episodes): {np.mean(steps_per_episode[-10:]):.2f}")

with open(f"{OUT}/exp2_output_log.txt", "w") as f:
    f.write("\n".join(log_lines))
print("\n\nDONE - Experiment 2 complete.")
```